

EduGeine-AI Documentation

AI-Powered Learning Assistant

Project Documentation and Final Submission Report

FastAPI • Google Gemini • HTML • CSS • JavaScript

Clear Project Editing & Maintenance Guide

This section explains what each important file controls, where it is located, what should normally be edited, and what should not be changed casually.

1. Explorer: where to edit each part

Location	Edit this when you want to change...	Do not change casually
main.py	API routes, validation, backend wiring	Endpoint names without updating frontend/tests
ai_client.py	Gemini model, API connection, retries	Real API key in source code
qna.py	Q&A prompt and answer rules	Gemini connection logic
explanation_module.py	Explanation prompt/local option	API key
quiz_module.py	Quiz format and validation	JSON rules without updating tests/frontend
summary_module.py	Summary prompt/output style	Gemini connection logic
learning_path.py	Learning-path prompt/stages	API key
templates/index.html	Page text, controls, result display	Backend route names casually
static/style.css	Colors, spacing, responsive design	API logic
tests/test_api.py	Automated tests and mocked responses	Live Gemini calls for routine tests
.env	Private local API configuration	Never submit/publish
.env.example	Safe environment-variable template	Real API key
requirements.txt	Python dependencies	Removing packages without checking imports

2. Safe way to edit the project

Make a backup before changing a working file. Make one change at a time and run the tests afterward.

Edit the file responsible for the feature. For example, change Q&A; wording in qna.py and visual design in static/style.css.

After saving, run:

```
python -m pytest
```

Your current verified baseline is 12 passed tests.

Then start the application and manually check the changed feature:

```
python -m uvicorn main:app --host 127.0.0.1 --port 8002
```

Do not manually edit .venv/, __pycache__/, or .pytest_cache/. They are local/generated folders.

3. Clear examples of common changes

Change visible page text: edit templates/index.html.

Change visual design: edit static/style.css.

Change Q&A; behavior: edit qna.py.

Change concept explanation: edit explanation_module.py.

Change quiz format: edit quiz_module.py and keep its validation consistent with the prompt.

Change summary behavior: edit summary_module.py.

Change learning recommendations: edit learning_path.py.

Change Gemini configuration: edit the private .env values, not the API key inside Python source.

```
GEMINI_API_KEY=YOUR_REAL_API_KEY
GEMINI_MODEL=gemini-3.6-flash
USE_LOCAL_EXPLANATION=false
```

Change automated tests: edit tests/test_api.py. Keep AI responses mocked for routine tests so testing does not consume live Gemini quota.

4. Final submission clarity

The final project should make the eight documented stages easy to identify: ideation, requirements, design, planning, development, testing, documentation, and demonstration.

Include in the final submission:

- Application Python files and modules
- templates/ and static/
- tests/test_api.py
- requirements.txt
- .env.example
- .gitignore
- README.md
- Final documentation PDF

Do NOT include:

- .env containing the real Gemini API key
- .venv/
- __pycache__/
- .pytest_cache/
- temporary/debug files

Version consistency note

The supplied original project PDF describes an earlier model configuration, while the completed implementation has been updated during development. The final documentation should describe the code and configuration actually being submitted.

Important Source Code Samples

This section includes selected code from the important project files. The samples are included so a reviewer can understand how the application is structured, how Gemini is connected, how quiz output is validated, and how the API is tested. The complete source files remain in the project submission.

1. main.py — FastAPI application

This is the central backend file. It creates the FastAPI application, serves the frontend, validates incoming text, and connects the five learning features to API endpoints.

```
from fastapi import FastAPI, HTTPException, Request
from fastapi.responses import HTMLResponse
from fastapi.staticfiles import StaticFiles
from fastapi.templating import Jinja2Templates
from pydantic import BaseModel, Field

from explanation_module import explain_concept
from qna import answer_question
from quiz_module import generate_quiz
from summary_module import summarize_text
from learning_path import get_learning_recommendations

app = FastAPI(
    title="EduGenie - AI Learning Assistant",
    version="1.0.0",
)

app.mount("/static", StaticFiles(directory="static"), name="static")
templates = Jinja2Templates(directory="templates")

class TextRequest(BaseModel):
    text: str = Field(..., min_length=1, max_length=30000)

@app.get("/", response_class=HTMLResponse)
async def home(request: Request):
    return templates.TemplateResponse(
        request=request,
        name="index.html",
    )

@app.get("/health")
async def health():
    return {"status": "ok", "service": "EduGenie"}

@app.post("/qa")
async def qa(payload: TextRequest):
    try:
        return {"answer": answer_question(payload.text)}
    except Exception as exc:
        raise HTTPException(status_code=500, detail=str(exc))

@app.post("/explain")
async def explain(payload: TextRequest):
    try:
        return {"explanation": explain_concept(payload.text)}
    except Exception as exc:
        raise HTTPException(status_code=500, detail=str(exc))

@app.post("/quiz")
async def quiz(payload: TextRequest):
    try:
        return {"quiz": generate_quiz(payload.text)}
    except Exception as exc:
        raise HTTPException(status_code=500, detail=str(exc))

@app.post("/summarize")
async def summarize(payload: TextRequest):
    try:
        return {"summary": summarize_text(payload.text)}
    except Exception as exc:
        raise HTTPException(status_code=500, detail=str(exc))

@app.post("/learn/recommendations")
async def recommendations(payload: TextRequest):
    try:
        return {"recommendations":
            get_learning_recommendations(payload.text)}
    except Exception as exc:
        raise HTTPException(status_code=500, detail=str(exc))
```

2. ai_client.py — Gemini integration

This module is responsible for communicating with Google Gemini. The API key is loaded from the private .env file rather than being written directly into the source code. The client is cached and temporary service failures are retried.

```
import os
import time
from functools import lru_cache

from dotenv import load_dotenv
from google import genai
from google.genai import types

load_dotenv()

MODEL_NAME = os.getenv("GEMINI_MODEL", "gemini-3.6-flash")
API_KEY = os.getenv("GEMINI_API_KEY") or os.getenv("GOOGLE_API_KEY")

@lru_cache(maxsize=1)
def get_client():
    if not API_KEY:
        raise RuntimeError(
            "Gemini API key is missing. "
            "Set GEMINI_API_KEY in your .env file."
        )
    return genai.Client(api_key=API_KEY)

def generate_text(prompt: str) -> str:
    if not prompt or not prompt.strip():
        raise ValueError("Prompt cannot be empty.")

    client = get_client()

    config = types.GenerateContentConfig(
        automatic_function_calling=types.FunctionCallingConfig(
            disable=True
        )
    )

    last_error = None

    for attempt in range(3):
        try:
            response = client.models.generate_content(
                model=MODEL_NAME,
                contents=prompt.strip(),
                config=config,
            )

            text = getattr(response, "text", None)

            if not text:
                raise RuntimeError("Gemini returned an empty response.")

            return text.strip()

        except Exception as exc:
            last_error = exc
            error_text = str(exc)

            if ("503" in error_text or "UNAVAILABLE" in error_text
                or "429" in error_text
                or "RESOURCE_EXHAUSTED" in error_text):
                if attempt < 2:
                    time.sleep(2 ** attempt)
                    continue

            raise

    raise RuntimeError(
        f"Gemini request failed after retries: {last_error}"
    )
```

Security note

The real GEMINI_API_KEY must never be included in the documentation, README, screenshots, or final ZIP. Use .env.example only as a safe template.

3. quiz_module.py — Quiz generation and validation

The quiz module requests exactly three multiple-choice questions and validates the returned structure. Each question must contain four options and an answer that matches one of those options.

```
import json
import re
from typing import Any

from ai_client import generate_text

def clean_json_block(text: str) -> str:
    text = text.strip()
    text = re.sub(r"^```(?:json)?\s*", "", text, flags=re.IGNORECASE)
    text = re.sub(r"\s*```$", "", text)
    return text.strip()

def _validate_quiz(data: Any) -> list[dict]:
    if not isinstance(data, list):
        raise ValueError("Quiz response must be a JSON list.")

    validated = []

    for item in data[:3]:
        question = str(item.get("question", "")).strip()
        options = item.get("options", [])
        answer = str(item.get("answer", "")).strip()

        if (not question or not isinstance(options, list)
            or len(options) != 4 or not answer):
            raise ValueError(
                "Each question requires a question, four options, and an answer."
            )

        options = [str(option).strip() for option in options]

        if answer not in options:
            raise ValueError("Answer must match one of the options.")

        validated.append({
            "question": question,
            "options": options,
            "answer": answer,
        })

    if len(validated) != 3:
        raise ValueError("Gemini must return exactly three questions.")

    return validated
```

Why this validation matters

Generative AI output is not guaranteed to follow a requested format. The validation layer prevents malformed quiz data from being passed to the frontend.

4. tests/test_api.py — Automated testing

The test suite verifies the API without repeatedly calling the live Gemini service. The AI-dependent functions are mocked during the AI endpoint tests. This makes the tests faster and avoids consuming Gemini API quota.

```
from unittest.mock import patch
from fastapi.testclient import TestClient

from main import app

client = TestClient(app)

def test_health():
    response = client.get("/health")
    assert response.status_code == 200
    data = response.json()
    assert data["status"] == "ok"
    assert data["service"] == "EduGenie"

def test_qa_ai_endpoint():
    with patch(
        "qna.generate_text",
        return_value="Photosynthesis is how plants make food."
    ):
        response = client.post(
            "/qa",
            json={"text": "What is photosynthesis?"}
        )

        assert response.status_code == 200
        data = response.json()
        assert "answer" in data
        assert isinstance(data["answer"], str)

# The complete project test file contains 12 tests.
# AI calls are mocked so tests do not consume Gemini quota.
```

Verified project result

The completed project test run collected 12 tests and reported **12 passed**. One non-blocking deprecation warning was reported by the installed Starlette/AnyIO test-client dependency.

Other important source files

qna.py — builds the educational Q&A; prompt and calls `generate_text()`.

explanation_module.py — generates beginner-friendly concept explanations and supports an optional local explanation model.

summary_module.py — summarizes educational text while preserving important facts and definitions.

learning_path.py — creates Beginner, Intermediate, and Advanced learning paths.

templates/index.html — provides the interactive browser interface and sends requests to the API.

static/style.css — controls the visual design, responsive layout, loading state, result cards, and quiz presentation.

Project Structure and File Organization

The following structure describes the folders and files used by the completed EduGenie-AI project. Each item has a specific role in the application and should be kept in the project root unless otherwise stated.

EduGenie-AI/	
.venv/	Virtual environment (local only)
static/	
style.css	Frontend styling
templates/	
index.html	Main web interface
tests/	
test_api.py	Automated API tests
main.py	FastAPI application and API routes
ai_client.py	Google Gemini client integration
qna.py	Question-and-answer feature
explanation_module.py	Concept explanation feature
quiz_module.py	Quiz generation and validation
summary_module.py	Text summarization feature
learning_path.py	Learning recommendations feature
requirements.txt	Python dependencies
.env	Private Gemini API configuration
.env.example	Safe configuration template
.gitignore	Files/folders excluded from version control
README.md	Project documentation

Required vs. Generated/Local Items

Item	Keep for submission?	Purpose
.venv/	No	Local Python environment; recreate with Python/pip.
__pycache__/	No	Python-generated cache files.
.pytest_cache/	No	Pytest-generated cache.
.env	No	Contains the private Gemini API key.
.env.example	Yes	Shows required environment variable names without the real key.
requirements.txt	Yes	Lists packages required to install the project.
main.py + modules	Yes	Core backend and AI feature implementation.
templates/ + static/	Yes	Frontend interface and styling.
tests/	Yes	Automated tests proving API behavior.
README.md	Yes	Project setup and usage documentation.

How the Project Folders and Files Are Created

The project can be created in Visual Studio Code from an empty folder. The folder structure separates the frontend, backend, AI integration, and automated tests so each responsibility is easy to locate.

1. Create the project folder

```
New-Item -ItemType Directory -Path "EduGenie-AI"\ncd "EduGenie-AI"
```

2. Create the main folders

```
New-Item -ItemType Directory -Path "templates"\nNew-Item -ItemType Directory -Path "static"\nNew-Item
```

3. Create the Python virtual environment

```
python -m venv .venv\n.\.venv\Scripts\Activate.ps1
```

The virtual environment keeps this project's Python packages separate from other Python projects on the computer.

4. Create the application files

Create the Python modules in the project root, then create index.html inside templates/ and style.css inside static/. Create test_api.py inside tests/. The configuration files .env, .env.example, .gitignore, requirements.txt, and README.md remain in the root.

5. Install the required software

Required: Python 3.11.x, Visual Studio Code, a modern web browser, and a Google Gemini API key. The project uses FastAPI for the backend, Uvicorn as the development server, Jinja2 for templates, python-dotenv for environment variables, Google GenAI for Gemini access, Pydantic for request validation, and pytest/httpx for automated testing.

6. Install Python dependencies

```
python -m pip install -r requirements.txt
```

Configuration, Running, and Testing

Environment configuration

Create a `.env` file in the project root. Do not place the real API key in `README.md`, `.env.example`, screenshots, or the submission ZIP. The `.env.example` file is safe to share because it contains only placeholder configuration.

```
GEMINI_API_KEY=YOUR_REAL_API_KEY
GEMINI_MODEL=gemini-3.6-flash
USE_LOCAL_EXPLANATION=false
```

Start the application

```
python -m uvicorn main:app --host 127.0.0.1 --port 8002
```

Open the displayed local address in the browser, for example `http://127.0.0.1:8002`. The application provides the five learning features through the web interface.

Available API endpoints

Method	Endpoint	Purpose
GET	/	Main EduGenie web interface
GET	/health	Service health check
POST	/qa	Question and answer
POST	/explain	Concept explanation
POST	/quiz	Three-question quiz generation
POST	/summarize	Educational text summarization
POST	/learn/recommendations	Structured learning recommendations

Automated testing

```
python -m pytest
```

The completed automated suite contains 12 tests. The AI endpoint tests use mocked AI responses, so running the test suite does not consume Gemini API quota. The successful run shown in the project evidence is 12 passed with one non-blocking deprecation warning from the installed `Starlette/AnyIO` test-client dependency.

Interactive API documentation

FastAPI also exposes interactive API documentation at `/docs` while the server is running. This can be used to inspect the available routes and request schemas.

Final Submission and Installation Checklist

Before submitting the project

1. Confirm all five web features work: Q&A;, Explain Concept, Quiz Generator, Summarize Text, and Learning Recommendations.
2. Run `python -m pytest` and confirm the test suite passes.
3. Keep the real `.env` file private and do not submit it.
4. Include `.env.example` with placeholder values.
5. Do not submit `.venv/`, `__pycache__/`, or `.pytest_cache/`.
6. Include `requirements.txt` so another computer can install the dependencies.
7. Include `README.md` with setup, installation, running, and testing instructions.
8. Include `templates/index.html` and `static/style.css`.
9. Include all Python modules and tests.
10. Review screenshots so they clearly show the application results and API documentation.
11. Keep the final documentation PDF with the project submission.

Installation summary for a new computer

1. Install Python 3.11.x and VS Code
2. Open the EduGenie-AI project folder
3. Create/activate `.venv`

This section is intended to make the project reproducible: a reviewer can understand what each folder is for, what must be installed, how the environment is configured, how the application is started, and how the automated tests are executed.

2. Definitions & Acronyms

AI — Artificial Intelligence — computer-based methods that perform tasks associated with reasoning, language understanding, or content generation.

NLP — Natural Language Processing — techniques for processing and generating human language.

LLM — Large Language Model — a language model capable of generating and transforming natural-language content from prompts.

Generative AI — AI that creates new content such as explanations, summaries, quizzes, or recommendations.

API — Application Programming Interface — an interface through which software components communicate.

REST — Representational State Transfer — an approach for web APIs using HTTP operations.

FastAPI — The Python web framework used for the EduGenie backend.

Uvicorn — The ASGI server used to run the FastAPI application.

Jinja2 — A Python HTML templating engine used by the frontend.

JSON — JavaScript Object Notation — structured data used for API exchange and quiz responses.

3. System Architecture & Core Modules

The application follows a modular request-and-response architecture. A user selects a learning task in the browser, the frontend sends the request to a FastAPI REST endpoint, the corresponding functional module prepares an educational prompt, the AI service generates the response, and the result is returned to the interface.

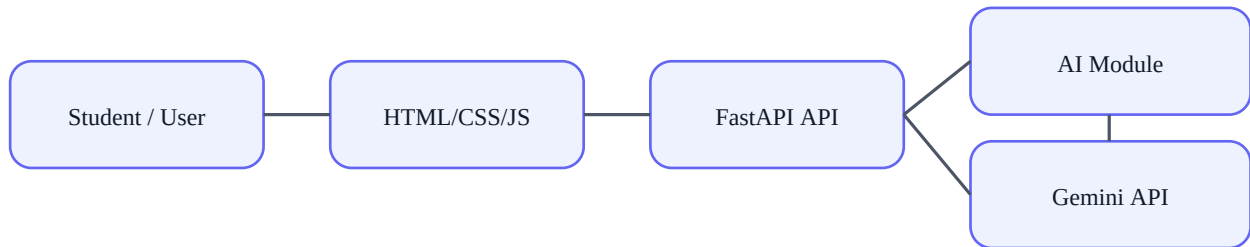


Figure A. EduGenie-AI high-level architecture flow.

Core Modules

- Q&A; Module — handles educational question answering.
- Explanation Module — produces beginner-friendly concept explanations; the local explanation path is optional and the current configuration can fall back to Gemini.
- Quiz Module — requests structured quiz data, removes Markdown code fences when necessary, parses JSON, and validates three questions with four options each.
- Summary Module — condenses educational passages while preserving important information.
- Learning Path Module — creates structured beginner, intermediate, and advanced learning recommendations.

Documented REST Endpoints

- GET /
- GET /health
- POST /qa
- POST /explain
- POST /quiz
- POST /summarize
- POST /learn/recommendations

4. Key Features & Important Points

- Single educational interface with task selection.
- AI-generated question answering and concept explanations.
- Three-question multiple-choice quiz generation.
- Educational text summarization with concise bullet-style output.
- Structured learning recommendations.
- Input validation for empty requests.
- REST API backend with clear endpoint separation.
- Responsive HTML/CSS frontend.
- Automated API testing with mocked AI responses, preventing test runs from unnecessarily consuming Gemini quota.

Security and Responsible Use

- The Gemini API key is stored in the local `.env` file and excluded from submission/version control.
- The submitted project should contain `.env.example` rather than the real `.env` file.
- Production deployment should additionally consider secret management, access control, rate limiting, privacy safeguards, and controlled logging.

5. Step-by-Step User Guide

1. Start the application

Activate the virtual environment, install requirements, configure the Gemini API key, and run the FastAPI application with Uvicorn.

2. Select a learning task

Choose Q&A, Explain Concept, Quiz Generator, Summarize Text, or Learning Recommendations from the task dropdown.

3. Enter a topic, question, or passage

Type the educational question, concept, topic, or text into the input area.

4. Generate the result

Click Generate and wait for the AI response to appear.

5. Review the result

Read the generated answer, explanation, quiz, summary, or learning path in the result section.

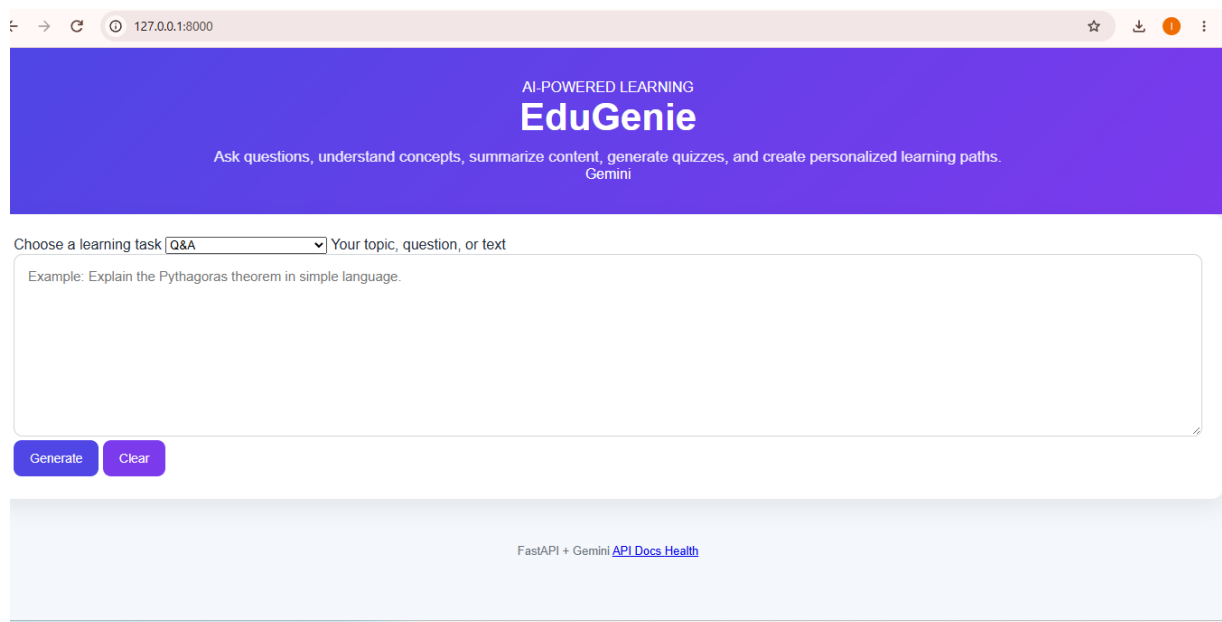


Figure 1. EduGenie-AI main user interface.

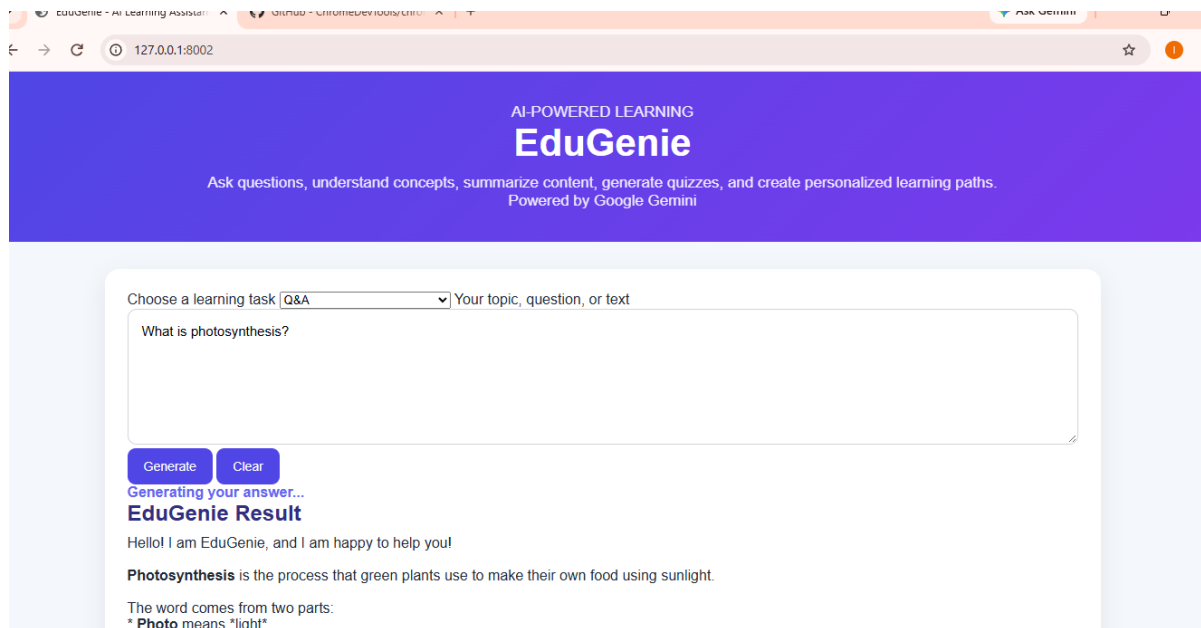


Figure 2. Question and Answer feature.

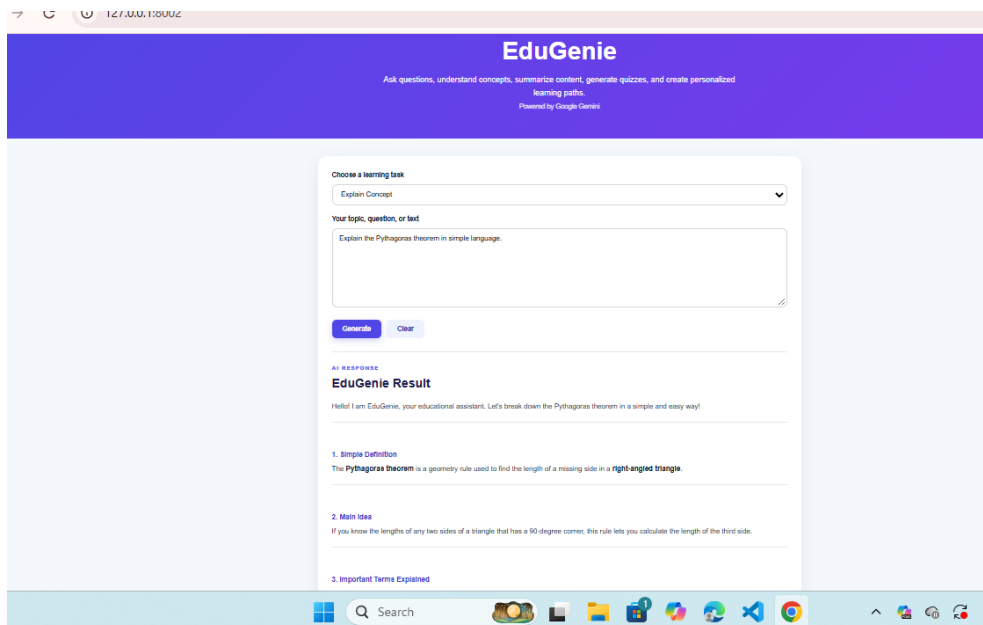


Figure 3. Concept Explanation feature.

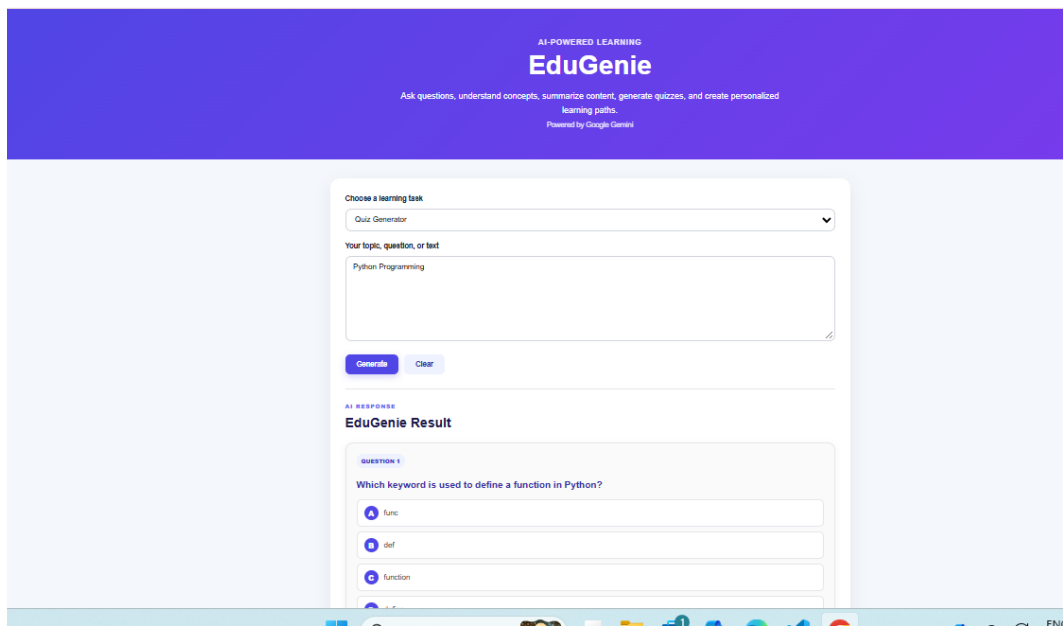


Figure 4. AI-generated quiz feature.

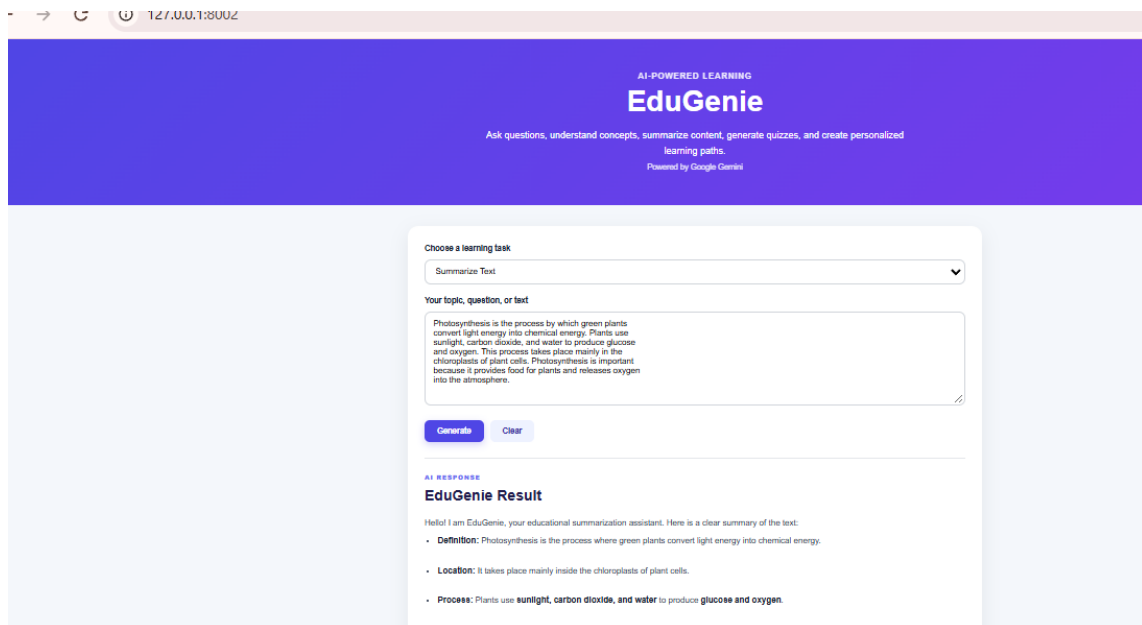


Figure 5. Educational text summarization feature.

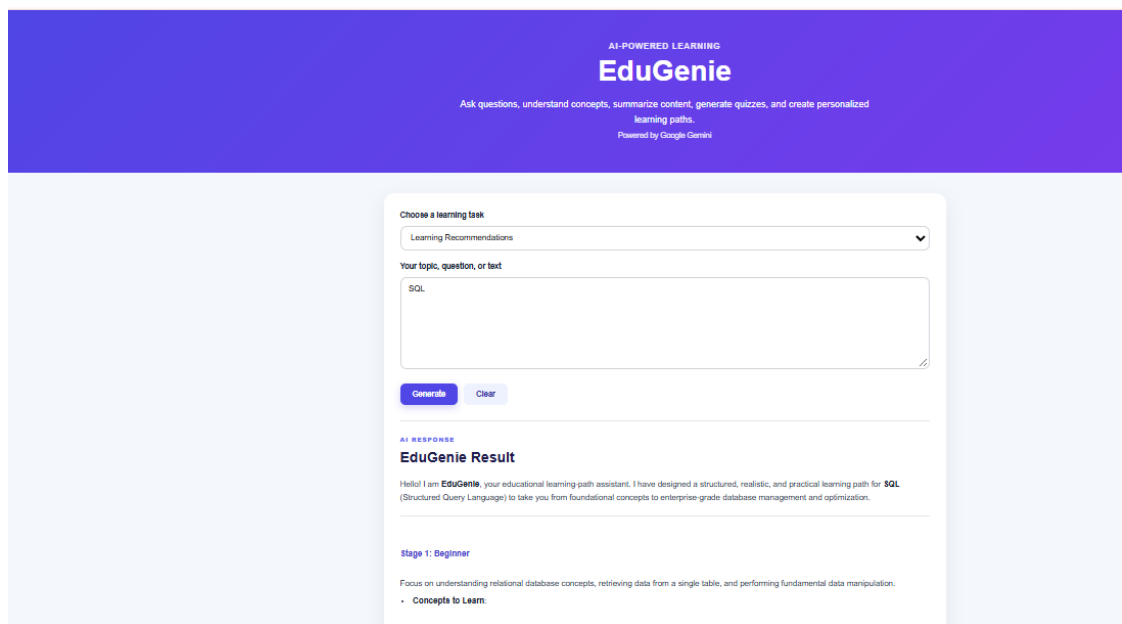


Figure 6. Learning Recommendations feature.

6. Technical Stack & Dependencies

Programming and Web Technologies

- Python 3.11
- FastAPI
- Uvicorn
- HTML, CSS, and JavaScript
- Jinja2
- Pydantic

AI Integration

- Google Gemini API through the Google GenAI Python SDK.
- The Gemini model is configurable through GEMINI_MODEL in the .env configuration.
- An optional local explanation model path is available through USE_LOCAL_EXPLANATION.

Testing

- Pytest
- FastAPI TestClient / HTTPX

Dependency Configuration

The project includes a requirements.txt file containing the application and testing dependencies, and a .env.example file containing the environment-variable template without exposing the real API key.

API Documentation Evidence

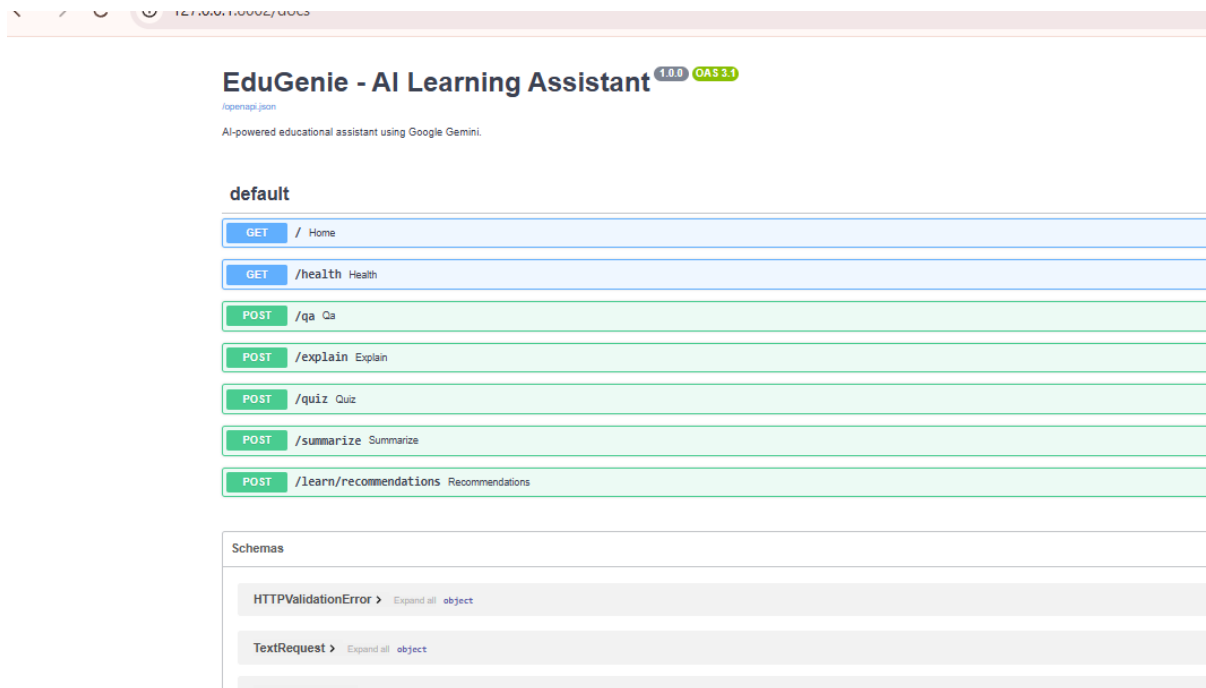


Figure 7. FastAPI Swagger documentation showing the available application endpoints.

7. Project Development & Workflow

Milestone 1 — Ideation and Architecture

The project was structured around a single educational assistant capable of supporting multiple learning tasks.

Milestone 2 — Core Functionality

Separate modules were created for question answering, explanations, quiz generation, summarization, and learning recommendations.

Milestone 3 — API Integration

FastAPI endpoints were connected to the functional modules and validated using request models.

Milestone 4 — Frontend Integration

The HTML/CSS/JavaScript interface was connected to the backend endpoints and configured to display generated results and errors clearly.

Milestone 5 — Testing and Production Readiness

The automated test suite was expanded to cover both validation behavior and AI endpoint behavior using mocked AI responses. This allows repeatable testing without consuming live Gemini quota.

8. Project Testing & Evidence

Automated Testing

The project test suite was executed using:

```
python -m pytest
```

Recorded result: 12 tests collected and 12 tests passed. One dependency deprecation warning was reported; it did not cause a test failure.

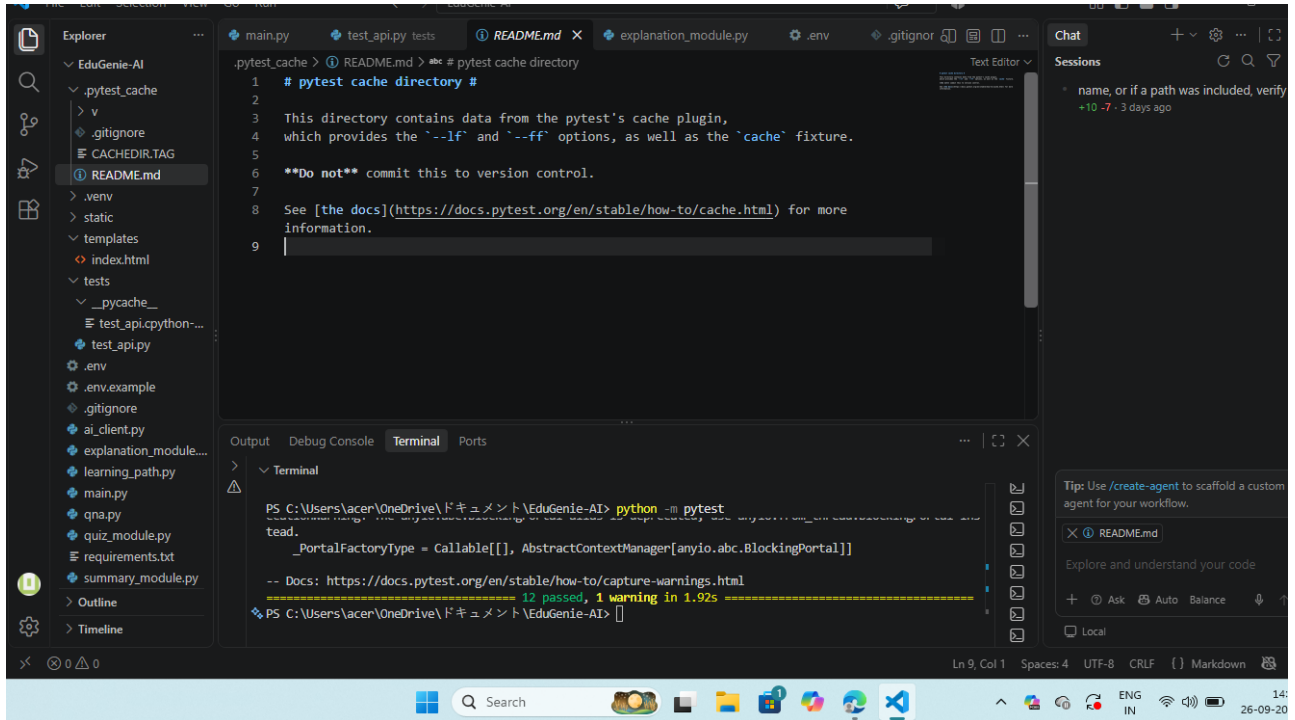


Figure 8. Automated Pytest result showing 12 tests passed.

Functional Testing Areas

- Question answering — verified through the Q&A; interface.
- Concept explanation — verified through the Explain Concept interface.
- Quiz generation — verified through the Quiz Generator interface.
- Summarization — verified through the Summarize Text interface.
- Learning recommendations — verified through the Learning Recommendations interface.

Test Design

The automated AI endpoint tests use mocked generate_text responses. This verifies endpoint behavior, response structure, quiz validation, and error handling without repeatedly calling the live Gemini service.

Screenshot Evidence & Explanation

The screenshots below are included as practical evidence of the implemented EduGenie-AI features. Each explanation describes what the screenshot demonstrates and how it relates to the application workflow.

Figure 1 — EduGenie-AI main user interface

This screenshot shows the main application screen. The user can select a learning task, enter a topic or question, and use the Generate button to send the request to the appropriate FastAPI endpoint. The Clear button resets the input and result area.

Figure 2 — Question and Answer feature

This screenshot demonstrates the Q&A; workflow. A student enters an educational question, the request is processed by the Q&A; module, and the generated answer is displayed in the EduGenie Result area in a readable format.

Figure 3 — Concept Explanation feature

This screenshot demonstrates the Explain Concept task. The user supplies a concept and receives a beginner-friendly explanation. The feature is intended to break difficult ideas into clearer language and useful examples.

Figure 4 — AI-generated quiz feature

This screenshot demonstrates quiz generation. EduGenie creates multiple-choice questions with answer options from the supplied topic or text. The result is displayed as an interactive quiz-style output.

Screenshot Evidence & Explanation — Continued

Figure 5 — Educational text summarization feature

This screenshot demonstrates the Summarize Text task. A longer educational passage is submitted and EduGenie returns a shorter version that keeps the main educational information while removing unnecessary repetition.

Figure 6 — Learning Recommendations feature

This screenshot demonstrates the Learning Recommendations task. EduGenie creates a structured learning path, progressing from beginner concepts toward more advanced learning and practical activities.

Figure 7 — FastAPI Swagger documentation

This screenshot provides API evidence through the FastAPI Swagger interface. It shows the available routes, including health checking and the Q&A, explanation, quiz, summarization, and learning-recommendation endpoints. This confirms that the backend exposes the documented REST interface.

Figure 8 — Automated Pytest result

This screenshot shows the project test run in the VS Code terminal. The recorded result is 12 tests collected and 12 tests passed. A dependency deprecation warning is also visible; it is non-blocking and did not cause any test failure. The automated suite uses mocked AI responses for repeatable endpoint testing without repeatedly consuming live Gemini quota.

Testing evidence note: The screenshot in Figure 8 is the automated test evidence used in this report. It supports the documented result of 12 passing tests and shows the project test command executed from the EduGenie-AI project directory.

9. Phase-Wise Project Submission

Phase 1 — Brainstorming & Ideation

Problem definition, target users, proposed solution, objectives, and expected benefits.

Phase 2 — Requirement Analysis

Functional, non-functional, and technical requirements.

Phase 3 — Project Design

Architecture, modules, APIs, frontend design, and folder structure.

Phase 4 — Project Planning

Milestones, development activities, risks, and deliverables.

Phase 5 — Project Development

Implementation and integration of backend, AI modules, REST APIs, and frontend.

Phase 6 — Project Testing

Automated validation and functional testing evidence.

Phase 7 — Project Documentation

Final technical report, screenshots, testing evidence, and project instructions.

Phase 8 — Project Demonstration

Demonstration video and any required public sharing link, if required by the institution.

Final Submission Package

- Source code and project folders.
- requirements.txt.
- .env.example.
- .gitignore.
- README.md.
- tests/test_api.py.
- Final project documentation PDF.
- Do not submit .env, .venv, __pycache__, or .pytest_cache.

10. Future Roadmap

- Persistent student accounts and learning history.
- Teacher or educator dashboard.
- Detailed learning-progress analytics.
- Additional AI model providers.
- Retrieval-augmented generation using a verified educational knowledge base.
- Multilingual learning support.
- Stronger moderation and safety controls.
- Cloud deployment and scalable infrastructure.
- Monitoring, caching, and improved observability.

11. Conclusion

EduGenie-AI demonstrates a practical generative-AI educational assistant that combines a FastAPI backend, a responsive web interface, modular educational features, and Google Gemini integration. The completed application supports question answering, concept explanation, quiz generation, summarization, and learning recommendations.

The project has been functionally demonstrated through the supplied screenshots and has an automated test suite with 12 passing tests. The final submission should preserve the working source code, include the documentation and testing evidence, and exclude private environment files and local development caches.

Final Submission Checklist

- ✓ Application source code included
- ✓ Frontend templates and CSS included
- ✓ requirements.txt included
- ✓ .env.example included
- ✓ .gitignore included
- ✓ README.md included
- ✓ Automated tests included
- ✓ 12 tests passing
- ✓ Final PDF documentation included
- ✓ Real .env file excluded
- ✓ .venv and cache folders excluded from the submission ZIP